

Week 1: Requirement Analysis and Customer Database Module

1. Requirement Analysis

Understanding the Travel Booking System

A travel booking platform allows customers to search for destinations, explore available travel packages, select suitable packages, make bookings, complete payments, and manage their travel details. A database is required to maintain important information about customers, destinations, travel packages, bookings, payments, and reviews. Properly organized data helps the system manage customer activities and travel transactions efficiently.

The Process Flow

- **Destination Search:** The customer explores available destinations and searches for a suitable travel location.
- **Package Selection:** The customer views available travel packages and selects the required package.
- **Booking Confirmation:** The customer provides the necessary travel details and confirms the booking.
- **Payment:** The customer selects an available payment method and completes the payment process.
- **Travel Confirmation:** Once the booking and payment are completed, the travel booking is confirmed.
- **Feedback and Assistance:** After the trip, the customer can provide feedback or contact support regarding the booking or travel experience.

Entity	Description	Key Attributes	Relationships
Customer	Stores details of customers who book travel packages.	Customer_id (PK), First_name, Last_name, Email, Phone, Created_at	One-to-Many with Booking and Review
Travel Agency	Stores details of travel agencies that provide travel packages.	Agency_id (PK), Agency_name, Email, Phone, Rating, Bank_details	One-to-Many with Travel Package
Travel Package	Stores details of travel packages available for booking.	Package_id (PK), Agency_id (FK), Package_name, Description, Price, Available_seats, Destination	Many-to-One with Travel Agency, One-to-Many with Review
Booking	Stores details of bookings made by customers.	Booking_id (PK), Customer_id (FK), Booking_date, Status, Total_amount, Travel_date	Many-to-One with Customer, One-to-One with Payment
Payment	Stores payment information for customer bookings.	Payment_id (PK), Booking_id (FK), Payment_method, Status, Transaction_id, Amount, Payment_date	One-to-One with Booking
Review	Stores ratings and comments given by customers about travel packages.	Review_id (PK), Customer_id (FK), Package_id (FK), Rating, Comment, Created_at	Many-to-One with Customer and Travel Package

Customer Table Creation

The Customer table is created to maintain the basic details of customers who use the travel booking system. It stores important information such as customer ID, first name, last name, email, phone number, and the date on which the customer record was created.

```
SQL> create table customer(customer_id varchar2(20) Primary key, first_name varchar2(20), last_name varchar2(20), email varchar2(30), phone number(10), created_at date);
```

Describe Customer Table

The Customer table is used to check the structure of the table, including the column names, data types, and other details of the customer information stored in the Travel Booking Management System.

```
SQL> desc customer
Name                               Null?      Type
-----
CUSTOMER_ID                       NOT NULL   VARCHAR2(20)
FIRST_NAME                        VARCHAR2(20)
LAST_NAME                         VARCHAR2(20)
EMAIL                             VARCHAR2(30)
PHONE                             NUMBER(10)
CREATED_AT                        DATE
```

Insert Values

The insert command is used to add customer information into the Customer table. Multiple customer records can be entered into the table with details such as customer ID, name, email, phone number, and created date.

```
SQL> insert into customer values('C001', 'Arun','Kumar','arun@gmail.com',9876543210, date '2026-01-10');
1 row created.

SQL> insert into customer values('C002', 'Priya','Sharma','priya@gmail.com',9876543211, date '2026-01-15');
1 row created.

SQL> insert into customer values('C003', 'Rahul','Raj','rahul@gmail.com',9876543212, date '2026-02-05');
1 row created.

SQL> insert into customer values('C004', 'Divya','Suresh','divya@gmail.com',9876543213, date '2026-02-12');
1 row created.
```

Display Customer Records

The stored customer information can be viewed using the select command

```
SQL> select*from customer;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME
EMAIL	PHONE	CREATED_A
C001	Arun	Kumar
arun@gmail.com	9876543210	10-JAN-26

C002	Priya	Sharma
priya@gmail.com	9876543211	15-JAN-26
C003	Rahul	Raj
rahul@gmail.com	9876543212	05-FEB-26

CUSTOMER_ID	FIRST_NAME	LAST_NAME
EMAIL	PHONE	CREATED_A
C004	Divya	Suresh
divya@gmail.com	9876543213	12-FEB-26

C005	Karthik	Mohan
karthik@gmail.com	9876543214	01-MAR-26
C006	Sneha	Ravi
sneha@gmail.com	9876543215	08-MAR-26

CUSTOMER_ID	FIRST_NAME	LAST_NAME
EMAIL	PHONE	CREATED_A
C007	Vishnu	Kumar
vishnu@gmail.com	9876543216	20-MAR-26

C008	Ananya	Krishnan
ananya@gmail.com	9876543217	02-APR-26
C009	Naveen	Raj
naveen@gmail.com	9876543218	15-APR-26

CUSTOMER_ID	FIRST_NAME	LAST_NAME
EMAIL	PHONE	CREATED_A
C010	Meena	Devi
meena@gmail.com	9876543219	01-MAY-26

10 rows selected.

Update

The UPDATE command is used to modify existing customer information in the Customer table. In this example, the phone number of customer C005 is changed to 9123456780 using the customer_id in the WHERE condition. The command successfully updates one row.

```
SQL> UPDATE customer
  2  SET phone = 9123456780
  3  WHERE customer_id = 'C005';

1 row updated.
```